

Resolución del ejercicio 1

1. *Captura del email*
2. *Procesamiento inicial*
3. *Clasificación con IA*
4. *Validación y decisión*
5. *Creación de evento en Calendar*
6. *Creación de borrador en Gmail*
7. *Registro en Google Sheets*
8. *Evitar reprocesamientos*
9. *Alerta opcional*



Resultado esperado del *workflow*

```
Gmail Trigger
↓
Procesar Email
↓
Verificar si el Message ID ya existe en Google Sheets
↓
IF: ¿Ya fue procesado?
├── Sí → Detener flujo
└── No → Clasificar con IA
    ↓
    Validar respuesta de IA
    ↓
    Decidir acción
    ↓
    Crear evento en Calendar, si corresponde
    ↓
    Crear borrador en Gmail, si corresponde
    ↓
    Registrar resultado en Google Sheets
    ↓
    Enviar alerta opcional, si la prioridad es ALTA
```

1. Captura del email

El primer nodo del *workflow* es Gmail *Trigger*.
Este nodo detecta nuevos mensajes recibidos en la bandeja de entrada.

Configuración sugerida:

```
{
  "event": "messageReceived",
  "filters": {
    "labelIds": ["INBOX"],
    "includeSpamTrash": false
  },
  "simplify": true
}
```

Datos mínimos que debe recuperar el flujo:

- ID del mensaje
- ID del hilo
- Fecha
- Remitente
- Nombre del remitente
- Asunto
- Cuerpo o vista previa del mensaje
- Indicación de adjuntos

Así, el *workflow* trabaja solo con *emails* nuevos.

2. Procesamiento inicial

Después del *Gmail Trigger*, se agrega un nodo *Code* llamado “Procesar *Email*”. Su función es ordenar los datos del mensaje y dejarlos en una estructura clara para los pasos siguientes.

Código sugerido:

```
const email = $json;

function extractEmail(value = '') {
  const match = value.match(/<(.*?)>/);
  return match ? match[1] : value;
}

function extractName(value = '') {
  const match = value.match(/^"?(["<"]+)"?\s*</);
  if (match) return match[1].trim();
}
```

...

```
const extractedEmail = extractEmail(value);
return extractedEmail.split('@')[0] || 'Remitente';
}

function cleanHtml(text = '') {
  return text
    .replace(/<style[\s\S]*?<\style>/gi, ' ')
    .replace(/<script[\s\S]*?<\script>/gi, ' ')
    .replace(/<[^>]+>/g, ' ')
    .replace(/\s+/g, ' ')
    .trim();
}

const from = email.from || email.sender || '';
const rawBody =
  email.textPlain ||
  email.text ||
  email.body ||
  email.textHtml ||
  email.html ||
  email.snippet ||
  '';
```

...

...

```
return [  
  {  
    json: {  
      messageId: email.id || email.messageId || '',  
      threadId: email.threadId || '',  
      fecha: email.date || new Date().toISOString(),  
      senderEmail: extractEmail(from),  
      senderName: extractName(from),  
      asunto: email.subject || 'Sin asunto',  
      cuerpo: cleanHtml(rawBody).slice(0, 3000),  
      hasAttachments: Boolean(email.attachments?.length || email.hasAttachments || false)  
    }  
  }  
];
```

Salida esperada

```
{
  "messageId": "msg_001",
  "threadId": "thread_001",
  "fecha": "2026-10-01T10:30:00.000Z",
  "senderEmail": "cliente@email.com",
  "senderName": "Cliente",
  "asunto": "Reunión para revisar propuesta",
  "cuerpo": "Hola, me gustaría coordinar una reunión...",
  "hasAttachments": false
}
```

3. Clasificación con IA

Luego, el *workflow* envía el asunto y el cuerpo del *email* a un modelo LLM. El objetivo es clasificar el mensaje y obtener una respuesta estructurada en JSON.

- **Nodo sugerido:** OpenAI / Claude / Gemini / HTTP Request hacia un LLM compatible
- **System prompt sugerido:** “Eres un asistente de operaciones que clasifica *emails* de atención. Debes analizar el asunto y el cuerpo del *email*, y responder únicamente con JSON válido”.
- **Categorías permitidas:** SOPORTE, VENTAS, REUNION, DOCUMENTACION, INFORMATIVO, IRRELEVANTE.
- **Prioridades permitidas:** ALTA, MEDIA, BAJA.

Devuelve esta estructura:

```
{
  "categoria": "",
  "prioridad": "",
  "resumen": "",
  "requiereRespuesta": true,
  "requiereEvento": false,
  "fechaReunion": null,
  "horaReunion": null,
  "duracionMinutos": null
}
```

Reglas:

- No inventes datos.
- Si no hay fecha u hora explícita, usa *null*.
- Solo marca *requiereEvento* como *true* si hay fecha y hora concretas.
- Si hay bloqueo, reclamo grave, urgencia operativa o necesidad de resolver hoy, usa prioridad ALTA.
- Si el *email* solicita cotización o información comercial, usa la categoría VENTAS.
- Si el *email* propone una reunión, usa la categoría REUNION.
- El resumen debe tener un máximo de 25 palabras.
- No agregues explicaciones fuera del JSON.

User prompt sugerido:

Clasifica este *email*:

- Remitente: `{{ $json.senderName }}`
`<{{ $json.senderEmail }}>`
- Asunto: `{{ $json.asunto }}`
- Contenido: `{{ $json.cuerpo }}`
- Tiene adjuntos: `{{ $json.hasAttachments }}`

Ejemplo de salida esperada del modelo:

```
{
  "categoria": "REUNION",
  "prioridad": "MEDIA",
  "resumen": "Solicita coordinar una reunión para
revisar una propuesta.",
  "requiereRespuesta": true,
  "requiereEvento": true,
  "fechaReunion": "2026-10-15",
  "horaReunion": "10:00",
  "duracionMinutos": 30
}
```

4. Validación y decisión

Después de recibir la respuesta de la IA, se agrega un nodo *Code* llamado “Validar IA y decidir acción”.

Este nodo cumple dos funciones:

- Validar que la IA haya devuelto un JSON correcto.
- Definir qué acción debe realizar el *workflow*.

Código sugerido:

```
const original = $('Procesar Email').first().json;

const raw =
  $json.output ||
  $json.text ||
  $json.message ||
  $json.choices?.[0]?.message?.content ||
  '';

let parsed;

try {
  parsed = typeof raw === 'string' ? JSON.parse(raw) : raw;
} catch (error) {
  parsed = {};
}
```

...

```
const categoriasPermitidas = [
  'SOPORTE',
  'VENTAS',
  'REUNION',
  'DOCUMENTACION',
  'INFORMATIVO',
  'IRRELEVANTE'
];

const prioridadesPermitidas = ['ALTA', 'MEDIA', 'BAJA'];

const categoria = categoriasPermitidas.includes(parsed.categoria)
  ? parsed.categoria
  : 'INFORMATIVO';

const prioridad = prioridadesPermitidas.includes(parsed.prioridad)
  ? parsed.prioridad
  : 'BAJA';

const requiereRespuesta = Boolean(parsed.requiereRespuesta);
const requiereEvento = Boolean(parsed.requiereEvento);
```

...

...

```
const tieneDatosDeReunion =  
  parsed.fechaReunion &&  
  parsed.horaReunion &&  
  requiereEvento;  
  
let accion = 'REGISTRAR';  
  
if (categoria === 'REUNION' && tieneDatosDeReunion) {  
  accion = 'CREAR_EVENTO';  
} else if (requiereRespuesta && categoria !== 'IRRELEVANTE') {  
  accion = 'CREAR Borrador';  
}  
  
if (prioridad === 'ALTA' && accion === 'REGISTRAR') {  
  accion = 'REVISAR';  
}
```

...

...

```
return [  
  {  
    json: {  
      ...original,  
      categoria,  
      prioridad,  
      resumen: parsed.resumen || 'No se pudo generar un resumen confiable.',  
      requiereRespuesta,  
      requiereEvento,  
      fechaReunion: parsed.fechaReunion || null,  
      horaReunion: parsed.horaReunion || null,  
      duracionMinutos: parsed.duracionMinutos || 30,  
      accion,  
      borradorCreado: 'NO',  
      eventoCreado: 'NO',  
      estado: 'pendiente'  
    },  
  }  
];
```

Acciones posibles:

- crear_evento
- crear_borrador
- registrar
- revisar

Criterio de decisión:

- Si hay reunión con fecha y hora → crear_evento
- Si requiere respuesta, pero no evento → crear_borrador
- Si no requiere respuesta → registrar
- Si la prioridad es alta y no hay acción clara → revisar

5. Creación de evento en Calendar

Si la acción definida es crear_evento, se agrega un nodo Google Calendar para crear el evento.

Configuración sugerida:

- Operación: `Create Event`
- Calendario: `Primary`
- Título: `Reunión: {{$json.asunto}}`
- Fecha y hora de inicio:

```
{{ $json.fechaReunion }}T{{ $json.horaReunion }}:00
```

- Fecha y hora de finalización:

```
Fecha y hora de inicio + {{$json.duracionMinutos}} minutos
```

- Asistente: `{{ $json.senderEmail }}`

Descripción:

```
Email original de {{$json.senderName}}  
Asunto: {{$json.asunto}}  
Resumen: {{$json.resumen}}
```

Si el *email* pide una reunión, pero no incluye fecha u hora concreta, el evento no debe crearse. En ese caso, el flujo debe pasar a la creación de un borrador solicitando la información faltante.

Después de crear el evento, se puede agregar un nodo Set para actualizar el campo:

```
{  
  "eventoCreado": "SI"  
}
```

6. Creación de borrador en Gmail

El *workflow* debe crear un borrador cuando el *email* requiere respuesta. No debe enviarlo automáticamente.

Primero, se agrega un nodo *Code* llamado “Preparar Borrador”.

```
const item = $json;

let cuerpo = '';

if (item.accion === 'CREAR_EVENTO') {
  cuerpo = `Hola ${item.senderName},

Gracias por tu mensaje. Confirmo la reunión solicitada para el día ${item.fechaReunion} a las
${item.horaReunion}.

Te envié la invitación correspondiente por calendario.

Saludos.`;
}
```

...

```
if (item.accion === 'CREAR Borrador' || item.accion === 'REVISAR') {  
  if (item.categoria === 'SOPORTE') {  
    cuerpo = `Hola ${item.senderName},
```

Gracias por contactarnos. Recibimos tu solicitud de soporte y la revisaremos a la brevedad.

Resumen del caso: \${item.resumen}

Si puedes compartirnos algún dato adicional, captura o detalle del error, nos ayudará a avanzar más rápido.

```
Saludos.`;  
}
```

```
if (item.categoria === 'VENTAS') {  
  cuerpo = `Hola ${item.senderName},
```

Gracias por tu interés. Recibimos tu consulta y podemos ayudarte con más información.

Para orientarte mejor, ¿podrías contarnos brevemente el alcance de lo que necesitas y el objetivo principal del proyecto?

Quedamos atentos.

```
Saludos.`;  
}
```

...

...

```
if (item.categoria === 'DOCUMENTACION') {  
  cuerpo = `Hola ${item.senderName},
```

Gracias por enviar la documentación. Confirmamos la recepción del mensaje.

Vamos a revisar el contenido y te responderemos si necesitamos información adicional.

```
Saludos.`;  
}
```

```
if (item.categoria === 'REUNION') {  
  cuerpo = `Hola ${item.senderName},
```

Gracias por tu mensaje. Podemos coordinar una reunión.

¿Podrías confirmarnos qué día y horario te resultan convenientes?

```
Saludos.`;  
}  
}
```

...

...

```
return [  
  {  
    json: {  
      ...item,  
      draftSubject: `RE: ${item.asunto}`,  
      draftBody: cuerpo || '',  
      borradorCreado: cuerpo ? 'SI' : 'NO'  
    }  
  }  
];
```

Luego, se agrega un nodo Gmail configurado como *Create Draft*.

Configuración conceptual:

Operación:

Create Draft

Para:

{{ \$json.senderEmail }}

Asunto:

{{ \$json.draftSubject }}

Cuerpo:

{{ \$json.draftBody }}

Thread ID:

{{ \$json.threadId }}

Reply To:

{{ \$json.messageId }}

Resultado esperado:

El email queda guardado como borrador en Gmail.
La respuesta no se envía automáticamente.
Una persona puede revisarla antes de enviarla.

7. Registro en Google Sheets

Todos los casos deben registrarse en la hoja Registro M4.

Columnas esperadas: Fecha procesamiento | Message ID | Remitente | Asunto | Categoría | Prioridad | Resumen | Acción| Borrador creado | Evento creado | Estado

Ejemplo de resultado:

Fecha procesamiento	Message ID	Remitente	Asunto	Categoría	Prioridad	Acción	Borrador creado	Evento creado	Estado
2026-10-01 10:30	msg_001	cliente@email.com	Reunión para revisar propuesta	REUNION	MEDIA	CREAR_EVENTO	SI	SI	pendiente
2026-10-01 10:45	msg_002	soporte@email.com	No puedo acceder al sistema	SOPORTE	ALTA	CREAR Borrador	SI	NO	pendiente

8. Evitar reprocesamientos

Antes de clasificar el *email* con IA, el *workflow* debe buscar el *Message ID* en Google Sheets.

Configuración conceptual:

```
Buscar en Google Sheets:  
Columna: Message ID  
Valor: {{$json.messageId}}
```

Luego, un nodo *IF decide*:

```
Si el Message ID ya existe → detener el flujo  
Si el Message ID no existe → continuar con la clasificación
```

Este control evita respuestas duplicadas, eventos repetidos y registros innecesarios.

9. Alerta opcional

Si se implementó la alerta opcional, debe activarse solo cuando la prioridad sea ALTA.

La notificación puede incluir:

- Remitente
- Asunto
- Categoría
- Prioridad
- Resumen
- Acción sugerida

Casos de prueba mínimos

Caso 1: reunión con fecha y hora

Email:

Asunto: Reunión para revisar propuesta

“Hola, me gustaría coordinar una reunión para revisar la propuesta. ¿Podemos reunirnos el 15/10/2026 a las 10:00? Gracias”.

Resultado esperado:

- Categoría: REUNION
- Prioridad: MEDIA
- Acción: CREAR_EVENTO
- Evento creado: SI
- Borrador creado: SI
- Estado: pendiente

Caso 2: soporte urgente

Email:

Asunto: No puedo acceder al sistema

“Hola, desde esta mañana no puedo ingresar al sistema y necesito resolverlo hoy porque afecta mi operación. ¿Me pueden ayudar?”.

Resultado esperado:

- Categoría: SOPORTE
- Prioridad: ALTA
- Acción: CREAR_BORRADOR o REVISAR
- Evento creado: NO
- Borrador creado: SI
- Alerta opcional: SI
- Estado: pendiente

¡Terminaste el módulo!
Todo listo para rendir el examen